

A decorative graphic on the left side of the slide consists of a network of light blue lines and circles, resembling a circuit board or a neural network. The lines are of varying thickness and connect to small circles at various points, creating a complex, branching pattern that extends from the top to the bottom of the slide.

MITIGATING RISK IN CLOUD INFRASTRUCTURE

PREDICTING CAPACITY BREACHES IN CONVERGED SERVICES

MARTA WHEELER

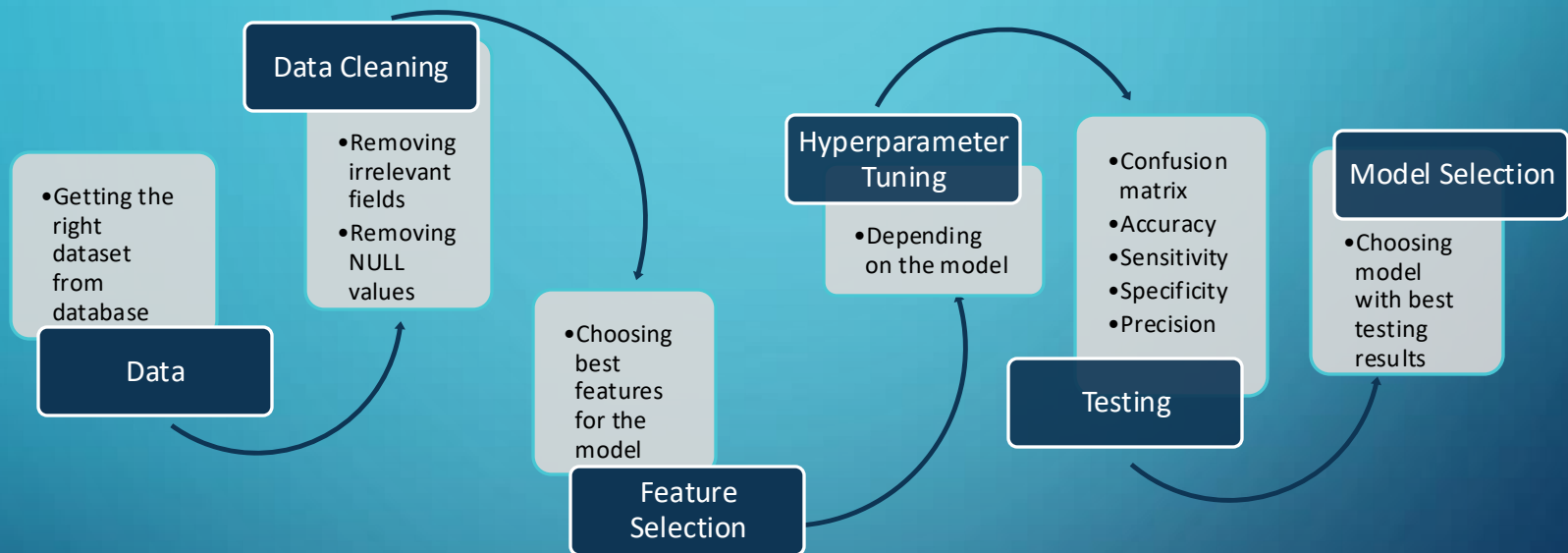
Table Of Contents

- Introduction
- Overview
- Data
- Methodology
- Results
- Conclusion

Overview

- Dataset acquisition (from database)
- Breach problem – predicting time and the reason of cluster breach in cloud infrastructure (global)
- Choosing data science methods to be implemented
- Dividing dataset into train and test
- Feature engineering – choosing the best features for each model
- Choosing the best models

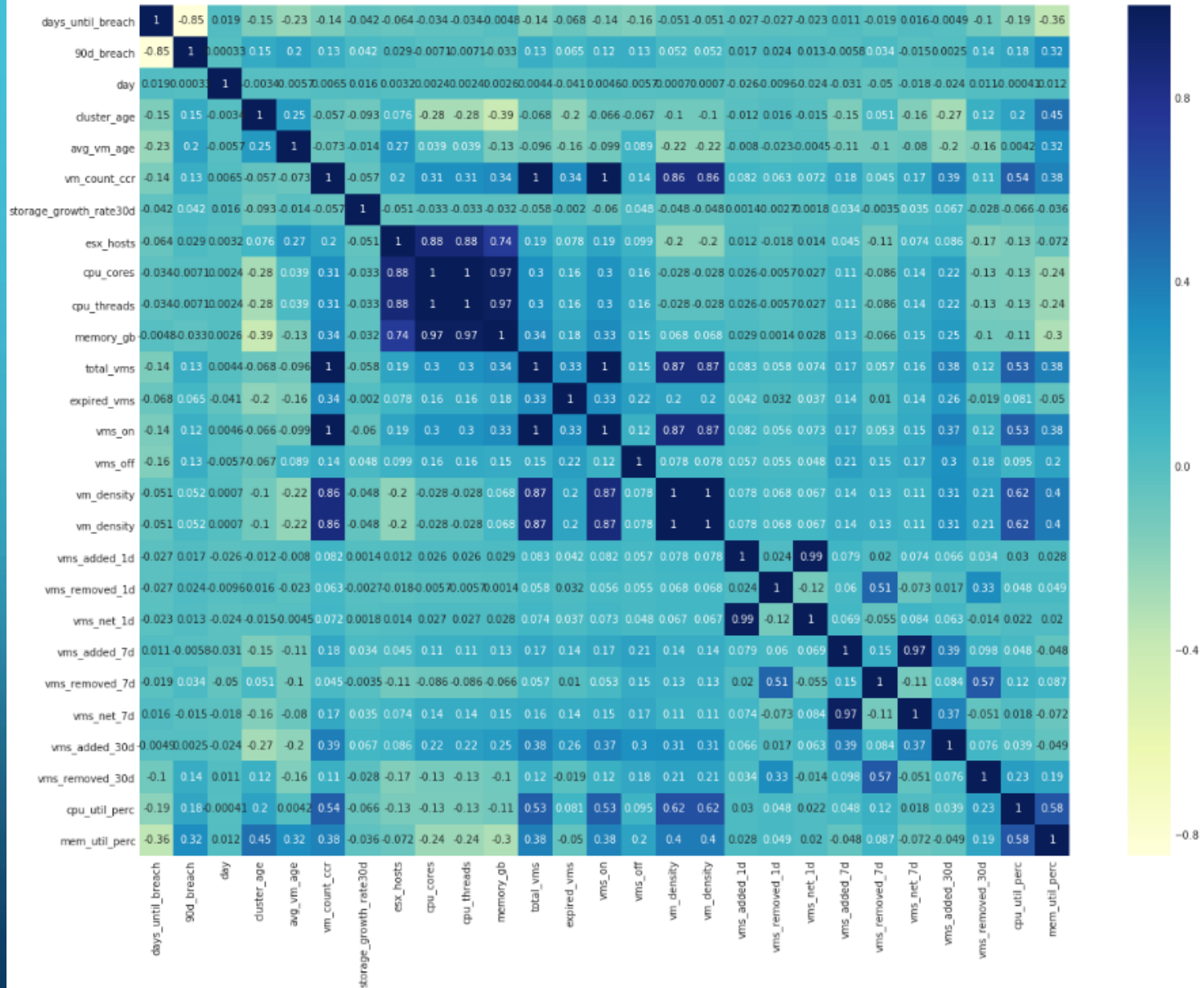
Process



Data

- Original Data Set :
 - 42.929 rows and 46 columns (8 categorical and 38 continuous)
 - 195 clusters, 246 dates
 - Breach probability 39.5 %
 - 90 day cluster breach probability 51.84 %
- After transformation (removing clusters after breach)
 - 4.978 rows, 46 columns
 - 51 clusters, 240 dates
 - 90 day cluster breach probability 62.11%
- After removing NULL values
 - 4.343 rows, 46 columns
 - 51 clusters, 197 dates
 - 90 day cluster breach probability 59.82%

Data - correlations between features



Problem

- Goal: Predicting cluster breach
- Solution: Examine state of cluster n days before the breach
- Independent variables – state of cluster before the breach (location, memory, cpu utilization etc.)
- Response variable:
 - days_until_breach – regression problem
 - 90d_breach – classification problem

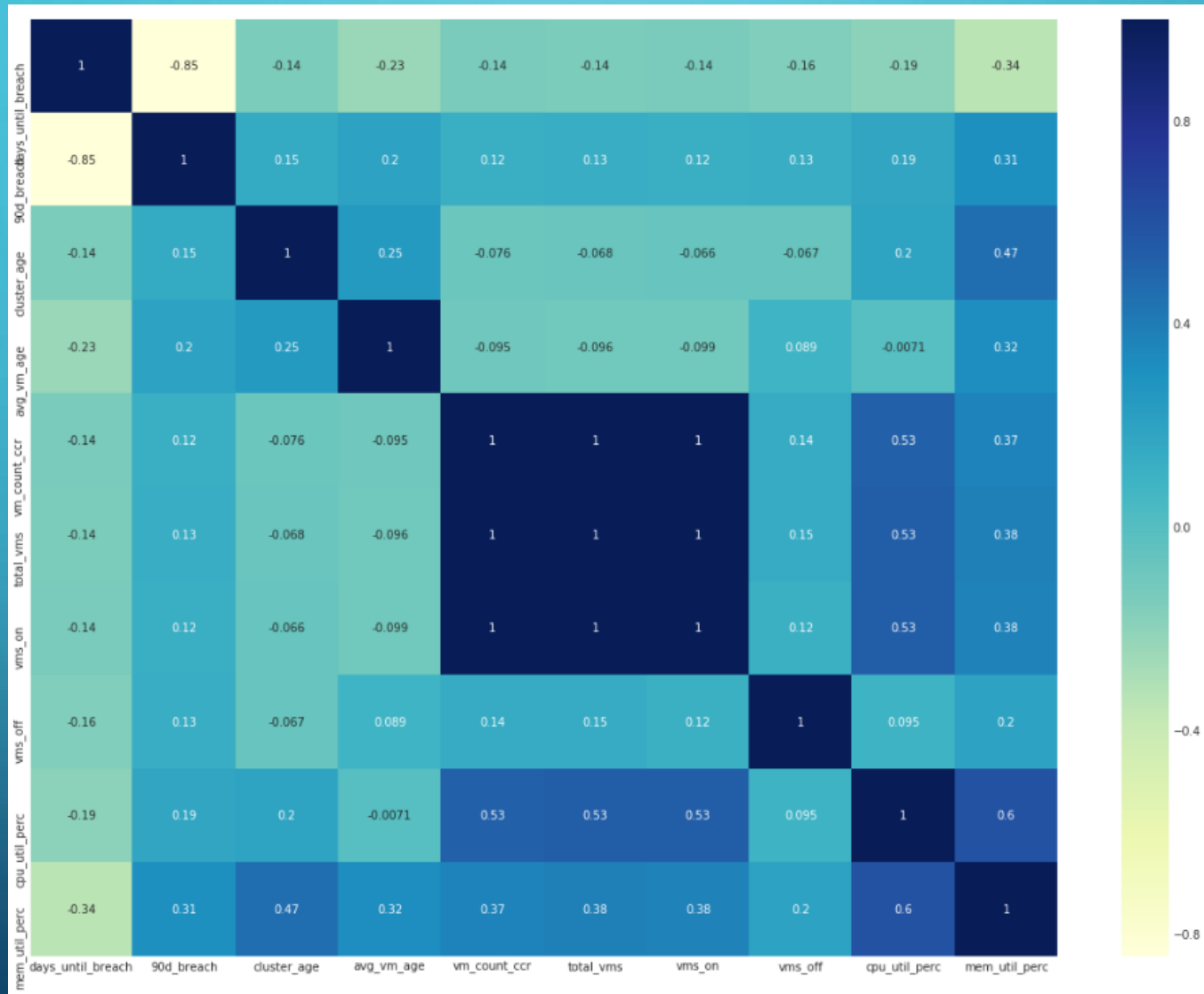
Methodology

- Regression Problem – predicting days_until_breach
 - Linear Regression
 - Decision Tree
- Classification Problem – predicting 90d_breach
 - Logistic Regression
 - Knn
 - Random Forest
 - Neural Network

Methodology

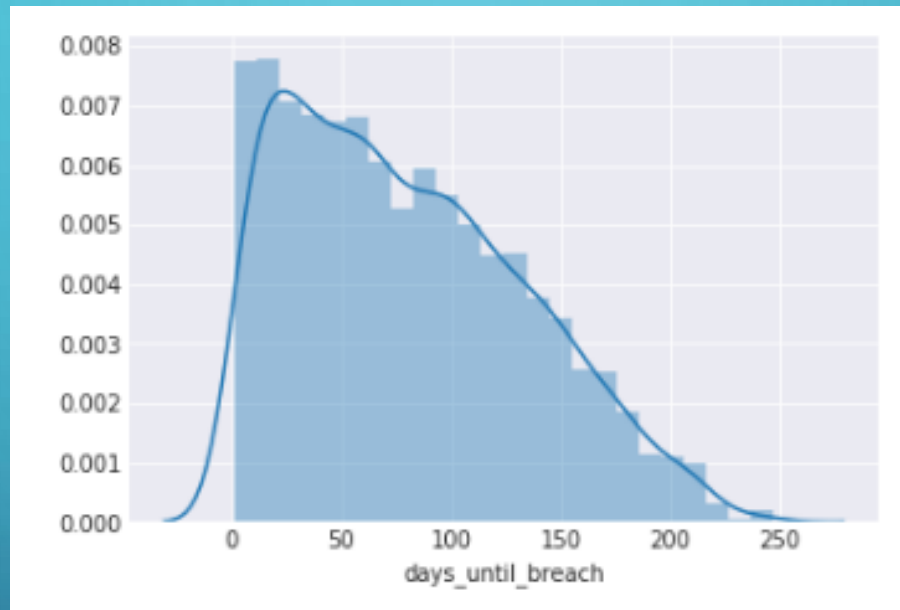
- Preliminary data analysis
- Feature engineering
- Fitting model
- Hyper parameters tuning
- Testing Model
- Choosing the best model

Preliminary Data Analysis



Preliminary Data Analysis

Response variable distribution



Preliminary Data Analysis

```
features.boxplot(column='mem_util_perc', by='90d_breach', showmeans=True)
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7ddc890>
```



```
features.boxplot(column='mem_util_perc', by='90d_breach', showmeans=True)
```

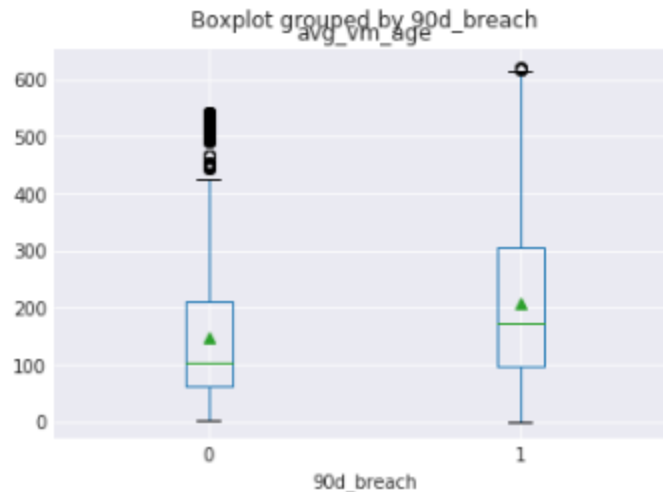
```
<matplotlib.axes._subplots.AxesSubplot at 0x7ddc890>
```



Preliminary Data Analysis

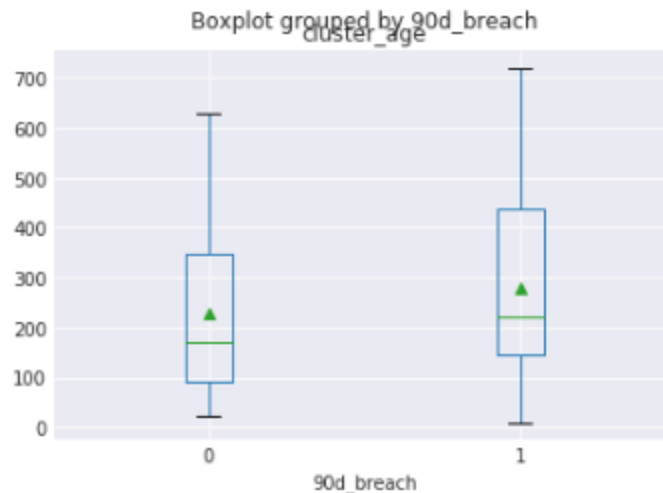
```
features.boxplot(column='avg_vm_age', by='90d_breach', showmeans=True)
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x86d2e90>
```



```
features.boxplot(column='cluster_age', by='90d_breach', showmeans=True)
```

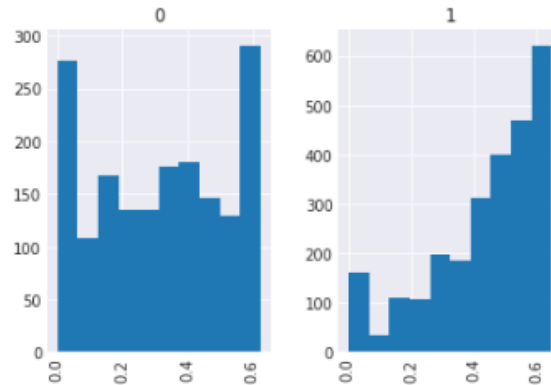
```
<matplotlib.axes._subplots.AxesSubplot at 0x86f7450>
```



Preliminary Data Analysis

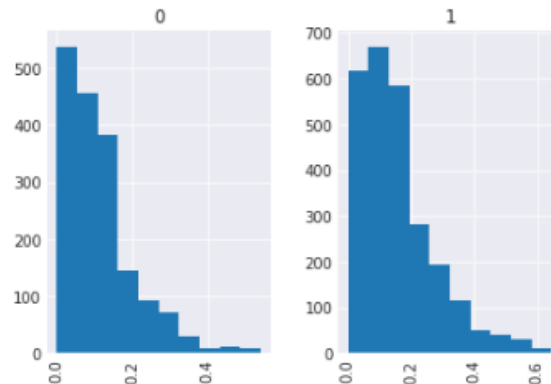
```
features.mem_util_perc.hist(by=features['90d_breach'])
```

```
array([<matplotlib.axes._subplots.AxesSubplot object at 0x892fa90>,  
      <matplotlib.axes._subplots.AxesSubplot object at 0x76349d0>],  
      dtype=object)
```



```
features.cpu_util_perc.hist(by=features['90d_breach'])
```

```
array([<matplotlib.axes._subplots.AxesSubplot object at 0x7631fd0>,  
      <matplotlib.axes._subplots.AxesSubplot object at 0x9292e90>],  
      dtype=object)
```

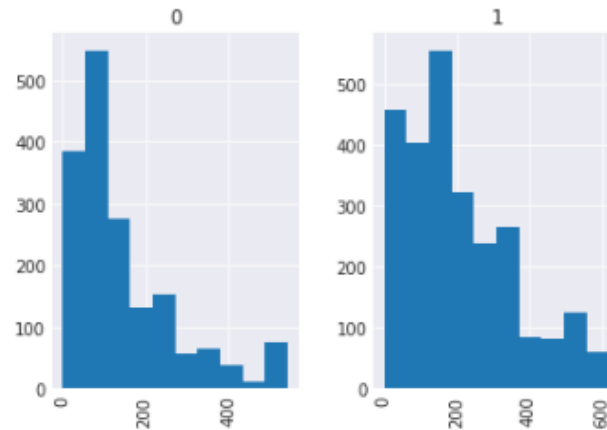


```
features.avg_vm_age.hist(by=features['90d_breach'])
```

Preliminary Data Analysis

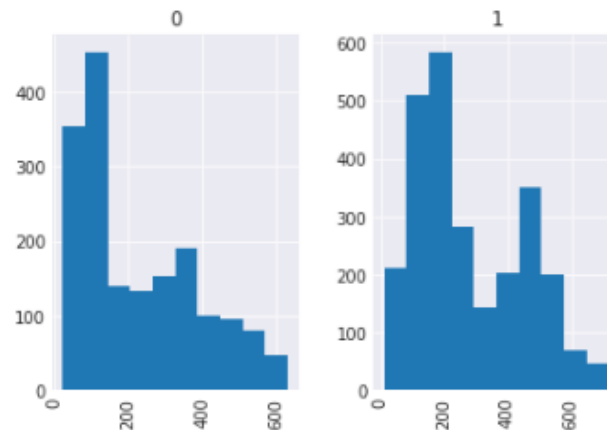
```
features.avg_vm_age.hist(by=features['90d_breach'])
```

```
array([<matplotlib.axes._subplots.AxesSubplot object at 0x8b64950>,  
      <matplotlib.axes._subplots.AxesSubplot object at 0x9494e10>],  
      dtype=object)
```



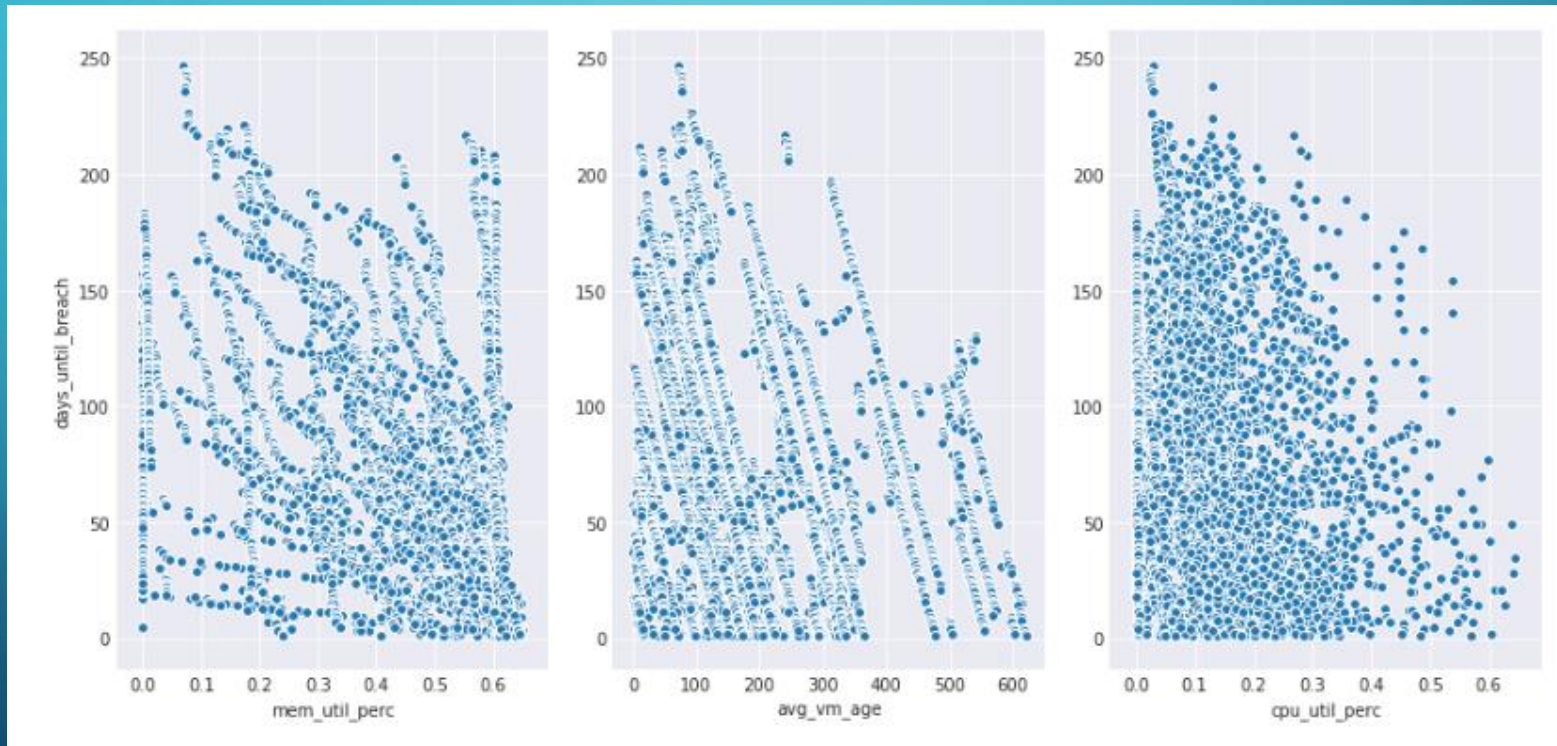
```
features.cluster_age.hist(by=features['90d_breach'])
```

```
array([<matplotlib.axes._subplots.AxesSubplot object at 0x948b610>,  
      <matplotlib.axes._subplots.AxesSubplot object at 0x9933290>],  
      dtype=object)
```

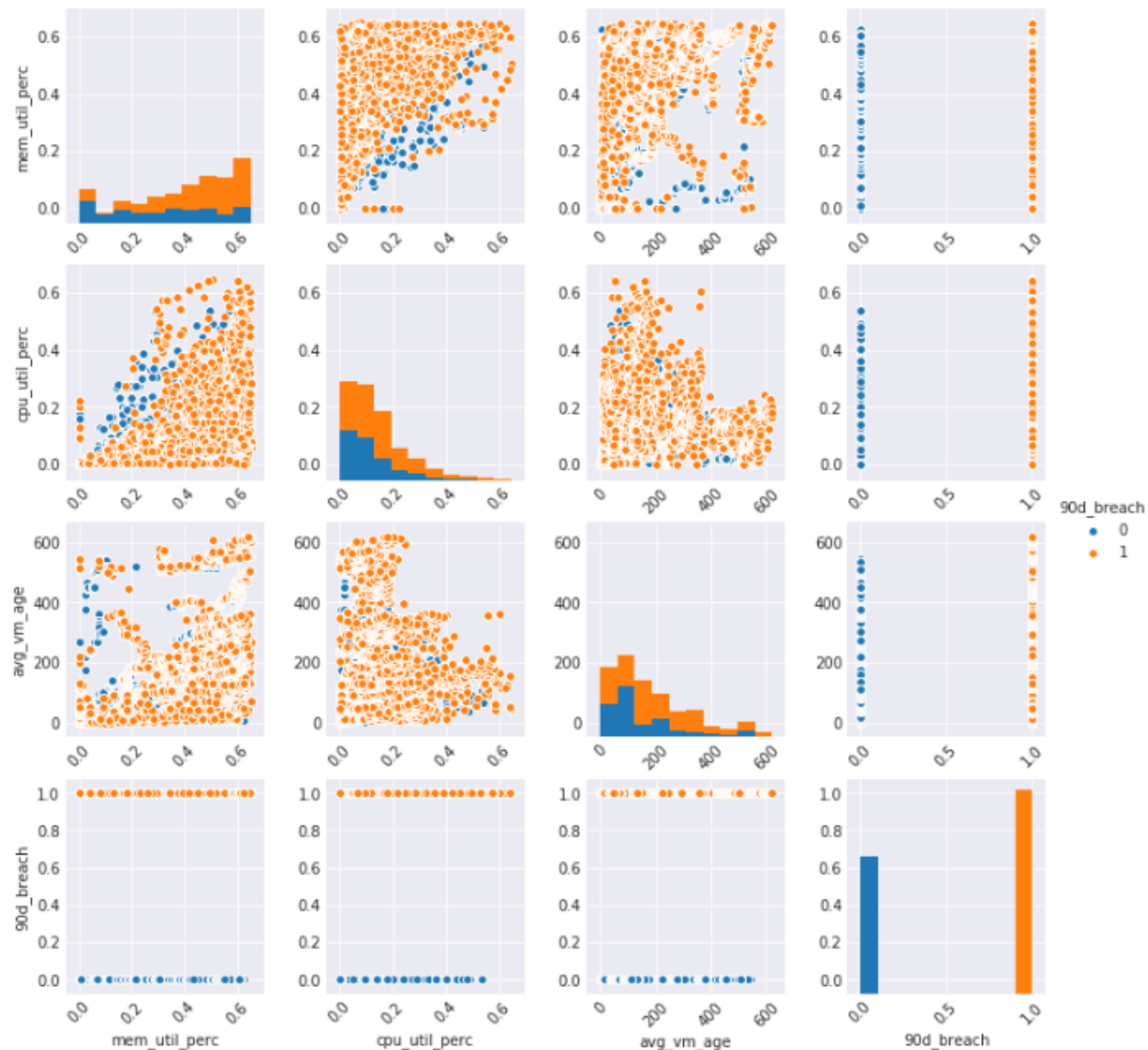


Data - Features

Continuous features – days until breach

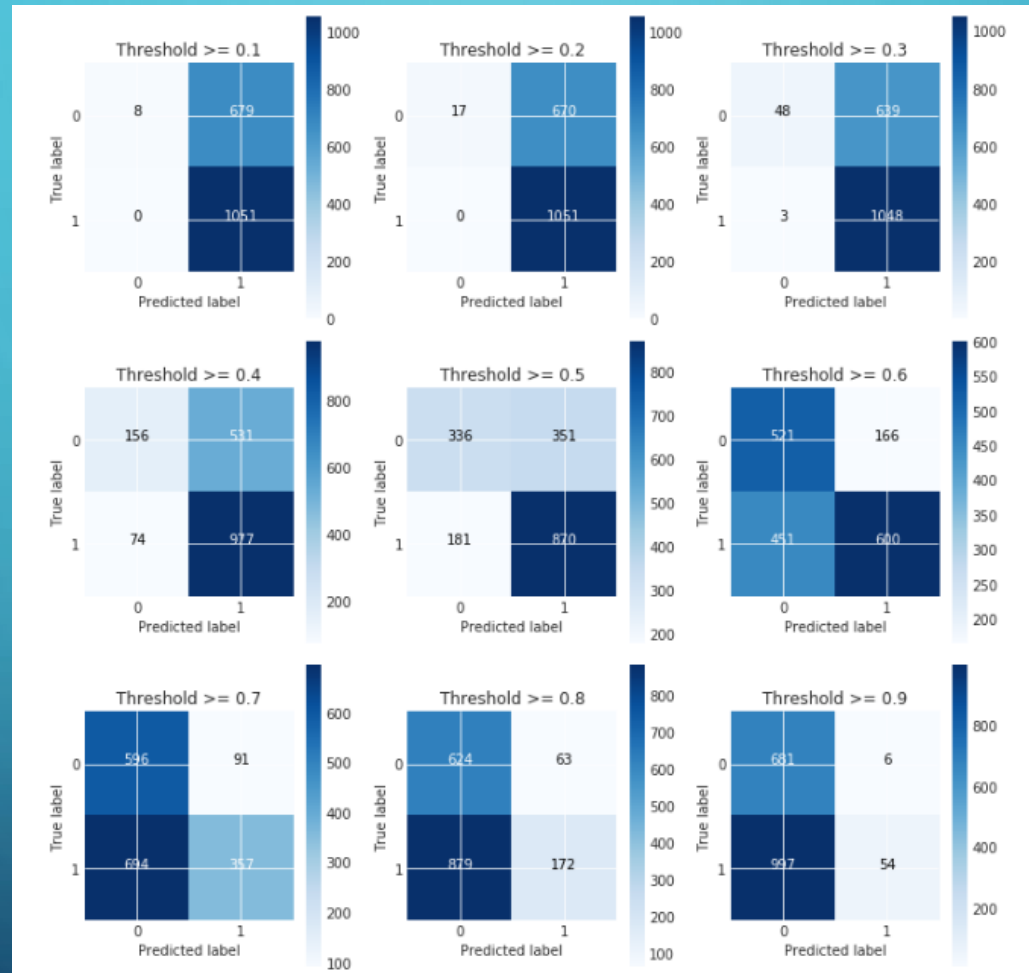


Data - Features



Parameters tuning

Logistic regression – choosing threshold

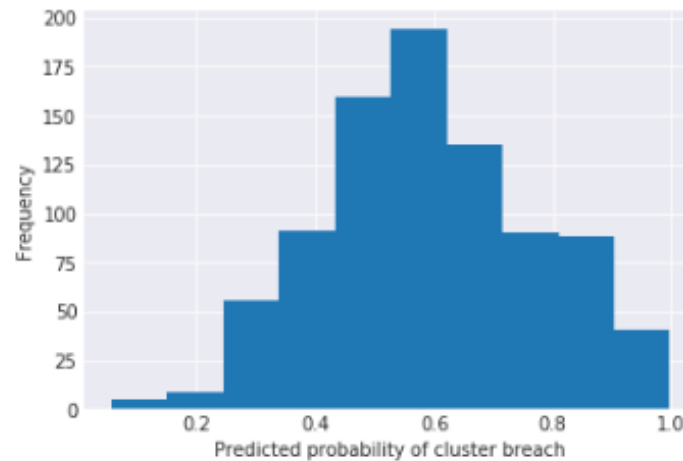


Parameters tuning

Logistic regression – choosing threshold

```
# plot the predicted probabilities
import matplotlib.pyplot as plt
plt.hist(y_pred_prob)
plt.xlabel('Predicted probability of cluster breach')
plt.ylabel('Frequency')
```

Text(0,0.5,u'Frequency')

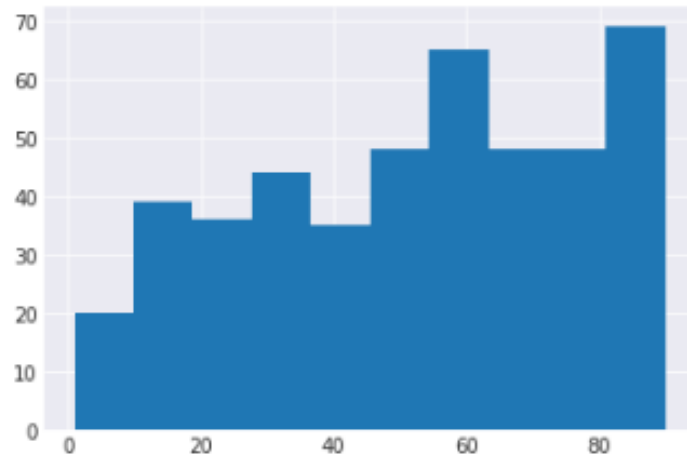


Prediction verification

```
wrong_pred = df_subset.days_until_breach[(df_subset.y_pred_all == 0) & (df_subset['90d_breach']==1)]
```

```
plt.hist(wrong_pred)
```

```
(array([20., 39., 36., 44., 35., 48., 65., 48., 48., 69.]),  
 array([ 1.,  9.9, 18.8, 27.7, 36.6, 45.5, 54.4, 63.3, 72.2, 81.1, 90. ]),  
 <a list of 10 Patch objects>)
```



ANN

```
# Last variable in the dataset is the dependent variable we're trying to predict  
Y = dataset[:,43]
```

```
Y
```

```
array([1., 1., 1., ..., 1., 1., 1.])
```

```
model = Sequential()  
model.add(Dense(65, input_dim=43, activation='relu'))  
model.add(Dense(43, activation='relu'))  
model.add(Dense(21, activation='relu'))  
model.add(Dense(9, activation='relu'))  
model.add(Dense(1, activation='sigmoid'))
```

```
# Compile model  
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])  
  
# sgd = optimizers.SGD(lr=0.01, decay=1e-6, momentum=0.9, nesterov=True)  
  
#model.compile( loss = "binary_crossentropy", optimizer = sgd, metrics=['accuracy'])
```

```
# Fit the model  
model.fit(X, Y, epochs=300, batch_size=50)
```

```
Epoch 1/300  
4343/4343 [=====] - 0s 75us/step - loss: 6.4056 - acc: 0.5982  
Epoch 2/300
```

Data - Features

- # create X (features) and y (response)
- feature_cols =
['cluster_age','avg_vm_age','vm_count_ccr','storage_growth_rate30d','esx_hosts','cpu_cores','cpu_threads','memory_gb','total_vms','expired_vms','vms_on','vms_off','vm_density','cpu_util_perc','mem_util_perc']
- X16 = df_subset[feature_cols]
- y16 = df_subset['90d_breach']

Data - Features

- `# create X (features) and y (response)`
- `feature_cols8 = ['cluster_age','avg_vm_age','vm_count_ccr',
'total_vms','vms_on','vms_off','cpu_util_perc','mem_util_perc']`
- `X8 = df_subset[feature_cols8]`
- `y8 = df_subset['90d_breach']`
- `feature_cols4 =`
`['cluster_age','avg_vm_age','cpu_util_perc','mem_util_perc']`
- `X4 = df_subset[feature_cols4]`
- `y4 = df_subset['90d_breach']`

Model Comparison

Model	Measure	Feature #		
		16	8	4
Logistic Regression	Accuracy	69.85%	66.97%	67.32%
	Sensitivity/Recall	52.26%	42.66%	41.53%
	Specificity	81.94%	83.69%	85.05%
	Precision	66.55%	64.26%	65.63%
Knn(2)	Accuracy	98.04%	95.97%	92.87%
	Sensitivity/Recall	99.72%	97.74%	95.76%
	Specificity	96.89%	94.76%	90.87%
	Precision	95.66%	92.76%	87.82%
Knn(3)	Accuracy	97.81%	95.28%	92.87%
	Sensitivity/Recall	98.31%	94.35%	89.83%
	Specificity	97.46%	95.92%	96.99%
	Precision	96.40%	94.08%	91.12%
Knn(5)	Accuracy	97.58%	95.51%	91.02%
	Sensitivity/Recall	96.89%	94.07%	88.98%
	Specificity	98.06%	96.50%	92.43%
	Precision	97.17%	94.87%	88.98%
Decision Tree	Accuracy	97.35%	94.71%	71.69%
	Sensitivity/Recall	97.18%	94.07%	41.53%
	Specificity	97.48%	95.15%	92.43%
	Precision	96.36%	93.02%	79.03%
ANN	Accuracy	59.82%		
NULL	Accuracy	59.96%		

Best model

- Best model – Knn with $n=5$ and 16 features
- Confusion matrix

Predicted	0	1	All
True			
0	343	11	354
1	10	505	515
All	353	516	869

- Model detected 505 of 515 breaches
- Model falsely detected 10 breaches
- Model didn't detect 11 breaches

Next steps

- Feature engineering and hyperparameters tuning in more details
- Regression problem
 - Linear Regression
 - Decision Tree
 - ANN
- K-means – for all data
- K-means – to create vm maturity score – avg vm size
- Automation (connecting directly with the database - pyodbc)